# Compte rendu — TP Android : Quiz en images

| | |
|---|---|
| **Étudiant** | Obaid Allah — ISET Djerba |
| **Thème choisi** | Formes géométriques (triangle, carré, cercle, étoile, rectangle) |
| **Technologies** | Kotlin, interface XML, View Binding, Material Components |
| **Dépôt** | Voir le lien GitHub remis dans Google Classroom / sur la plateforme |

## 1. Interface et widgets utilisés

L'écran est défini dans `res/layout/activity_main.xml`. Il est composé de deux parties :

- un **bandeau fixe** en haut (il ne défile pas, donc le score reste toujours visible) ;
- une **zone défilante** (`ScrollView`) qui contient la question, l'image, les trois choix et les boutons.

| Widget | Identifiant | Rôle |
|---|---|---|
| `TextView` | `txt_titre` | Titre « Quiz des formes » |
| `TextView` | `txt_score` | Score affiché en haut à droite (`Score : 2/5`) |
| `ProgressBar` (horizontale, max 5) | `progress_quiz` | Nombre de **réponses données** |
| `TextView` | `txt_progression` | Même information en texte (« Réponses données : 2 sur 5 ») |
| `TextView` | `txt_numero` | « Question 2 sur 5 » |
| `ImageView` | `img_question` | Image de la question, `scaleType="fitCenter"` pour éviter toute déformation |
| `TextView` | `txt_question` | Énoncé de la question |
| `MaterialButton` ×3 | `btn_choix1` à `btn_choix3` | Les trois réponses possibles |
| `TextView` | `txt_feedback` | Message « Bonne réponse » / « Mauvaise réponse » |
| `MaterialButton` | `btn_suivant` | « Question suivante » ou « Voir le résultat » |
| `LinearLayout` | `groupe_resultat` | Écran final (score, message, bouton Rejouer) — dans le même layout |
| `MaterialButton` | `btn_rejouer` | Réinitialise la partie |

Une seule activité (`MainActivity`) gère tout le jeu : il n'y a ni fragment, ni Jetpack Compose, ni base de données, ni accès réseau.

## 2. Tableau des questions et association des images

Chaque question est un objet de la classe `Question` (`Question.kt`) :

| Champ | Type | Contenu |
|---|---|---|
| `texte` | `String` | Énoncé de la question |
| `imageRes` | `Int` (`@DrawableRes`) | Identifiant de ressource, par exemple `R.drawable.triangle` |
| `choix` | `Array<String>` | Exactement trois réponses (vérifié par `require` dans `init`) |
| `indexBonneReponse` | `Int` | Indice 0, 1 ou 2 de la bonne réponse |

| | | |
|---|---|---|
| `descriptionImage` | `String` | Texte d'accessibilité (`contentDescription`) |

`MainActivity` déclare le tableau `questions: Array<Question>` avec exactement cinq éléments :

| N° | Image (`res/drawable`) | Question | Choix | Bonne réponse |
|---|---|---|---|---|
| 1 | `triangle.xml` | Quel est le nom de la forme affichée ? | Un carré / **Un triangle** / Un cercle | 1 |
| 2 | `carre.xml` | Combien d'angles droits possède cette forme ? | **4** / 3 / 2 | 0 |
| 3 | `cercle.xml` | Combien de sommets possède cette forme ? | 4 / 1 / **0** | 2 |
| 4 | `etoile.xml` | Combien de branches compte cette étoile ? | 6 / **5** / 4 | 1 |
| 5 | `rectangle.xml` | Qu'est-ce qui distingue cette forme d'un carré ? | Elle a trois côtés / Elle a cinq côtés / **Ses côtés ne sont pas tous égaux** | 2 |

Les images sont des **drawables vectoriels XML** (noms en minuscules, sans espace ni accent). La bonne réponse occupe des positions différentes (1, 0, 2, 1, 2) pour ne pas être devinée.

## 3. Événements, score et progression

**Événements.** Les trois boutons partagent la même logique : dans `onCreate`, une boucle `boutonsChoix.forEachIndexed { index, bouton -> bouton.setOnClickListener { onChoixClique(index) } }` enregistre un écouteur qui transmet l'indice du bouton. `btn_suivant` appelle `onSuivantClique()` et `btn_rejouer` appelle `rejouer()`.

**Variables d'état.** `indexQuestion`, `score`, `nbReponses`, `dejaRepondu`, `choixSelectionne`, `partieTerminee`.

**Fonctions principales.**

| Fonction | Rôle |
|---|---|
| `afficherQuestion()` | Affiche image, texte et trois choix ; réactive les boutons |
| `onChoixClique(index)` | Valide la réponse, met à jour score et progression |
| `afficherCorrection()` | Message, marques ✓ / ✗, désactivation des boutons, bouton suivant |
| `majBandeau()` | Actualise score, `ProgressBar` et texte de progression |
| `onSuivantClique()` | Question suivante ou résultat final |
| `afficherResultatFinal()` | Masque les choix, affiche le score final et Rejouer |
| `rejouer()` | Remet tout à zéro |

**Calcul du score.** Dans `onChoixClique`, `nbReponses++` est exécuté dans tous les cas ; `score++` n'est exécuté que si `index == questions[indexQuestion].indexBonneReponse`.

**Score ≠ progression.** Le *score* compte les bonnes réponses ; la *progression* compte les réponses données. Une mauvaise réponse fait avancer la barre sans augmenter le score.

## 4. Réponses aux questions du compte rendu

**Comment représentez-vous une question et sa bonne réponse ?** Par la classe `Question` : un texte, une ressource image, un tableau de trois chaînes et l'indice (0, 1 ou 2) de la bonne réponse dans ce tableau. Le bloc `init` vérifie qu'il y a bien trois choix et un indice valide.

**Comment le clic sur un bouton permet-il de savoir quel choix a été sélectionné ?** Les boutons sont rangés dans la liste `boutonsChoix` dans l'ordre des choix. À l'enregistrement de l'écouteur, `forEachIndexed` fournit l'indice du bouton, transmis à `onChoixClique(index)`. Le bouton `i` correspond donc toujours à `choix[i]`.

**Comment empêchez-vous l'attribution de plusieurs points pour une question ?** Deux protections : la première ligne de `onChoixClique` est `if (dejaRepondu || partieTerminee) return`, et `afficherCorrection()` désactive les trois boutons (`isEnabled = false`). Le booléen `dejaRepondu` n'est remis à `false` qu'au passage à la question suivante.

**Quelle différence faites-vous entre le score et la progression ?** Le score est le nombre de bonnes réponses (`score`, affiché en haut à droite). La progression est le nombre de questions auxquelles on a répondu (`nbReponses`, affiché dans la `ProgressBar`). Elle atteint 5/5 même si toutes les réponses sont fausses.

**Comment détectez-vous la dernière question et affichez-vous le résultat ?** `indexQuestion == questions.lastIndex` : le bouton affiche alors « Voir le résultat ». Au clic, `onSuivantClique()` met `partieTerminee = true` et appelle `afficherResultatFinal()` au lieu d'incrémenter l'indice ; on ne sort donc jamais du tableau.

## 5. Tests

Les scénarios obligatoires de l'énoncé sont listés ci-dessous. La colonne « Résultat observé » est à remplir après exécution sur l'émulateur ou le téléphone, avec les captures correspondantes dans `captures/`.

| Test | Résultat attendu | Résultat observé |
|---|---|---|
| Lancer une nouvelle partie | Première question, trois choix, score 0/5, progression 0/5 | ☐ |
| Bonne réponse | Score +1 et progression +1 | ☐ |
| Mauvaise réponse | Score inchangé, progression +1 | ☐ |
| Appuyer plusieurs fois sur un choix | Aucun point supplémentaire ; les trois choix sont désactivés | ☐ |
| Question suivante | Image, texte et choix actualisés ; score conservé | ☐ |
| Cinq bonnes réponses | Score final 5/5, progression 5/5 | ☐ |
| Cinq mauvaises réponses | Score final 0/5, progression 5/5 | ☐ |
| Réponses mélangées | Score égal au nombre exact de réponses justes | ☐ |
| Rejouer | Retour à la première question, score et progression à zéro | ☐ |
| Petit écran | Tous les choix accessibles, textes lisibles | ☐ |
| Rotation de l'écran (facultatif) | La partie est conservée | ☐ |

**Difficultés rencontrées** *(à compléter avec votre expérience)* — points sensibles du projet : garder les boutons colorés lisibles une fois désactivés (un bouton désactivé est grisé par défaut : les couleurs sont donc appliquées explicitement) ; ne pas dépendre de la couleur seule (ajout de texte et de ✓ / ✗) ; restaurer l'état après rotation sans rejouer le calcul du score.